

C05-AT-2 (ASSESSMENT TOOL 14)

AAMEENA(192511284)

Q1. Complete Query Processing Pipeline

Pseudocode for the journey of an SQL statement from raw text to a result set, covering parsing, semantic validation, optimization, plan generation and execution.

```
ALGORITHM QueryProcessingPipeline(sql_query)
BEGIN
    // 1. LEXICAL & SYNTAX ANALYSIS (Parsing)
    tokens      = Lexer.tokenize(sql_query)
    parse_tree  = Parser.buildParseTree(tokens)    // grammar-based parse
    IF parse_tree is NULL THEN
        RAISE SyntaxError("Invalid SQL syntax")
    END IF

    // 2. SEMANTIC ANALYSIS
    valid = SemanticAnalyzer.check(parse_tree, Catalog)
    // verifies tables/columns exist, type compatibility,
    // privileges, aggregate/group-by correctness
    IF NOT valid THEN
        RAISE SemanticError("Unknown relation/attribute or type mismatch")
    END IF

    // 3. INTERMEDIATE REPRESENTATION
    logical_plan = Translator.toRelationalAlgebra(parse_tree)

    // 4. QUERY OPTIMIZATION
    candidate_plans = Optimizer.generateEquivalentPlans(logical_plan)
    FOR EACH plan IN candidate_plans DO
        plan.cost = CostEstimator.estimate(plan, Statistics)
    END FOR
    best_plan = MIN(candidate_plans BY cost)    // cost-based selection
```

```

// 5. CODE GENERATION
physical_plan = PlanGenerator.toExecutionPlan(best_plan)

// 6. EXECUTION
execution_engine = QueryExecutionEngine.load(physical_plan)
result_set = execution_engine.run()           // iterator/pipeline model

// 7. RESULT OUTPUT
RETURN Formatter.present(result_set)
END

```

Q2. Cost-Based Query Optimization — Join Ordering

Estimates and compares the I/O cost of alternative join orders for a multi-way join and selects the cheapest, dynamic-programming style.

```

ALGORITHM CostBasedJoinOrdering(Relations R1..Rn, Statistics S)
BEGIN
    // Base case: cost of accessing each single relation
    FOR EACH Ri IN {R1,...,Rn} DO
        bestPlan[{Ri}] = ScanPlan(Ri)
        bestCost[{Ri}] = S.pages(Ri)           // cost = number of disk pages
    END FOR

    // Build up plans for larger subsets (Selinger-style DP)
    FOR size = 2 TO n DO
        FOR EACH subset T OF {R1,...,Rn} WITH |T| = size DO
            bestCost[T] = INFINITY
            FOR EACH split (T1, T2) such that T1 UNION T2 = T, T1 INTERSECT T2 = {} DO
                FOR EACH joinMethod IN {NestedLoop, SortMerge, HashJoin} DO
                    cost = bestCost[T1] + bestCost[T2]
                        + JoinCost(joinMethod, bestPlan[T1], bestPlan[T2], S)
                    IF cost < bestCost[T] THEN
                        bestCost[T] = cost
                        bestPlan[T] = MakeJoinPlan(joinMethod, bestPlan[T1], bestPlan[T2])
                    END IF
                END FOR
            END FOR
        END FOR
    END FOR
END

```

```

        END IF
    END FOR
END FOR
END FOR
END FOR

RETURN bestPlan[{R1,...,Rn}], bestCost[{R1,...,Rn}]
END

FUNCTION JoinCost(method, planA, planB, S)
    // Example: Nested Loop I/O cost
    IF method = NestedLoop THEN
        RETURN planA.pages + (planA.rows * planB.pages)
    ELSE IF method = SortMerge THEN
        RETURN planA.pages + planB.pages + SortCost(planA) + SortCost(planB)
    ELSE // HashJoin
        RETURN 3 * (planA.pages + planB.pages) // build + probe (approx.)
    END IF
END FUNCTION

```

Q3. Nested Loop Join Algorithm

```

ALGORITHM NestedLoopJoin(OuterRelation R, InnerRelation S, condition  $\theta$ )
BEGIN
    result = EMPTY SET
    FOR EACH tuple r IN R DO // outer loop, block by block
        FOR EACH tuple s IN S DO // inner loop, re-scanned per outer tuple
            IF  $\theta(r, s)$  is TRUE THEN
                result = result UNION {r JOIN s}
            END IF
        END FOR
    END FOR
    RETURN result
END

```

Time Complexity Analysis (buffer pages & tuples)

- Let b_R , b_S = number of disk pages of R and S; n_R , n_S = number of tuples in R and S.
- Tuple-at-a-time Nested Loop: for every tuple of R, S is scanned fully $\rightarrow$ I/O cost = $b_R + (n_R \times b_S)$.
- Block Nested Loop (using B buffer pages, B-2 reserved for R, 1 for S, 1 for output):
I/O cost = $b_R + (\lceil b_R / (B-2) \rceil \times b_S)$ — fewer full scans of S.
- Best case: if R fits entirely in memory ($b_R \leq B-2$), cost reduces to $b_R + b_S$ (single pass over both).
- Time complexity in terms of tuples: $O(n_R \times n_S)$ comparisons in the worst case — quadratic — regardless of buffering strategy.

Q4. Deadlock Detection via Wait-For Graph

```
ALGORITHM DetectDeadlock(TransactionSet T, LockTable L)
```

```
BEGIN
```

```
    // Step 1: Build the Wait-For Graph (WFG)
```

```
    WFG = EMPTY GRAPH
```

```
    FOR EACH transaction  $T_i$  IN T DO
```

```
        WFG.addNode( $T_i$ )
```

```
    END FOR
```

```
    FOR EACH transaction  $T_i$  IN T DO
```

```
        FOR EACH resource r THAT  $T_i$  is WAITING FOR DO
```

```
             $T_j = L.getHolder(r)$     // transaction currently holding r
```

```
            IF  $T_j \neq \text{NULL}$  AND  $T_j \neq T_i$  THEN
```

```
                WFG.addEdge( $T_i \rightarrow T_j$ ) //  $T_i$  waits for  $T_j$ 
```

```
            END IF
```

```
        END FOR
```

```
    END FOR
```

```
    // Step 2: Cycle detection using DFS
```

```
    visited = SET(), recursionStack = SET()
```

```
    FOR EACH node N IN WFG.nodes DO
```

```
        IF N NOT IN visited THEN
```

```
    IF HasCycle(N, WFG, visited, recursionStack) THEN
        RETURN DEADLOCK_DETECTED, ExtractCycle(recursionStack)
    END IF
END IF
END FOR
RETURN NO_DEADLOCK
END
```

```
FUNCTION HasCycle(node, WFG, visited, recStack)
    visited.add(node)
    recStack.add(node)
    FOR EACH neighbour IN WFG.adjacent(node) DO
        IF neighbour IN recStack THEN
            RETURN TRUE          // back edge => cycle => deadlock
        ELSE IF neighbour NOT IN visited THEN
            IF HasCycle(neighbour, WFG, visited, recStack) THEN
                RETURN TRUE
            END IF
        END IF
    END FOR
    recStack.remove(node)
    RETURN FALSE
END FUNCTION
```

```
// Recovery: once a cycle is found, choose a victim (e.g. youngest transaction
// or lowest cost to abort), ROLLBACK it, and release its locks.
```

Q5. Basic Two-Phase Locking (2PL) Protocol

ALGORITHM TwoPhaseLocking(Transaction T)

BEGIN

state = GROWING // Phase 1: Growing Phase

lockSet = EMPTY SET

FOR EACH operation op IN T.operations DO

IF op requires lock on data item X THEN

IF state == SHRINKING THEN

RAISE ProtocolViolation("Cannot acquire lock after releasing one")

END IF

REQUEST lock(X, modeRequired(op))

WHILE lock(X) NOT GRANTED DO

T.status = WAITING

WAIT() // blocked until granted / deadlock check

END WHILE

lockSet.add(X)

T.status = ACTIVE

END IF

EXECUTE op // read/write on X

END FOR

// Phase 2: Shrinking Phase — begins at first unlock, no new locks after this

state = SHRINKING

FOR EACH X IN lockSet DO

RELEASE lock(X)

END FOR

COMMIT T

END

```

FUNCTION modeRequired(op)
  IF op.type == READ THEN RETURN SHARED
  IF op.type == WRITE THEN RETURN EXCLUSIVE
END FUNCTION

```

- Growing phase: locks may only be acquired, never released — guarantees serializability but not deadlock-freedom or recoverability by itself.
- Shrinking phase: locks may only be released, never acquired again.

Q6. Timestamp-Based Concurrency Control — Thomas' Write Rule

```

ALGORITHM ThomasWriteRule(Transaction Ti, DataItem X, Operation op)
BEGIN
  // Every transaction Ti has a unique timestamp TS(Ti) assigned at start
  // Every data item X carries: Read_TS(X) and Write_TS(X)

  IF op == READ(X) THEN
    IF TS(Ti) < Write_TS(X) THEN
      ROLLBACK(Ti)           // reading stale data not allowed
    ELSE
      EXECUTE READ(X)
      Read_TS(X) = MAX(Read_TS(X), TS(Ti))
    END IF

  ELSE IF op == WRITE(X) THEN
    IF TS(Ti) < Read_TS(X) THEN
      ROLLBACK(Ti)           // a younger Tx already read newer value
    ELSE IF TS(Ti) < Write_TS(X) THEN
      // Thomas' Write Rule: this write is obsolete because a later
      // transaction has already overwritten X. Instead of aborting,
      // simply IGNORE the write (do not physically apply it).
      IGNORE WRITE(X)
      CONTINUE Ti           // Ti proceeds normally
    ELSE
      EXECUTE WRITE(X)

```

```

    Write_TS(X) = TS(Ti)
  END IF
END IF
END

```

- Key distinction from basic timestamp ordering: an obsolete write ($TS(T_i) < Write_TS(X)$ but $TS(T_i) \geq Read_TS(X)$) is skipped rather than causing a rollback, improving concurrency.

Q7. ARIES Recovery Algorithm

```

ALGORITHM ARIES_Recovery(LogFile L, DataPages D)
BEGIN
  // ----- PHASE 1: ANALYSIS -----
  dirtyPageTable = LOAD_FROM_last_checkpoint(L)
  transactionTable = LOAD_FROM_last_checkpoint(L)
  startLSN = checkpoint.LSN

  FOR EACH logRecord FROM startLSN TO end_of_log DO
    IF logRecord.type == UPDATE THEN
      IF logRecord.pageID NOT IN dirtyPageTable THEN
        dirtyPageTable[pageID] = logRecord.LSN // recLSN
      END IF
      transactionTable[logRecord.transID].lastLSN = logRecord.LSN
    ELSE IF logRecord.type == COMMIT THEN
      transactionTable.remove(logRecord.transID)
    ELSE IF logRecord.type == BEGIN THEN
      transactionTable.add(logRecord.transID, status=ACTIVE)
    END IF
  END FOR

  losers = transactionTable.activeTransactions() // to be undone

  // ----- PHASE 2: REDO -----
  redoLSN = MIN(dirtyPageTable.recLSN values)
  FOR EACH logRecord FROM redoLSN TO end_of_log DO

```



```

IF logRecord.type == UPDATE THEN
    page = D.fetch(logRecord.pageID)
    IF page.pageLSN < logRecord.LSN THEN
        REDO(logRecord)          // repeat history, unconditionally
        page.pageLSN = logRecord.LSN
    END IF
END IF
END FOR

// ----- PHASE 3: UNDO -----
toUndo = PRIORITY_QUEUE(losers, ORDER BY lastLSN DESC)
WHILE toUndo NOT EMPTY DO
    logRecord = L.get(toUndo.pop())
    IF logRecord.type == UPDATE THEN
        UNDO(logRecord)          // apply Compensation Log Record (CLR)
        L.write(CLR(logRecord))
        IF logRecord.prevLSN != NULL THEN
            toUndo.push(logRecord.prevLSN)
        ELSE
            L.write(ABORT_COMPLETE, logRecord.transID)
        END IF
    END IF
END IF
END WHILE

RETURN DATABASE_CONSISTENT
END

```

Q8. Shadow Paging Recovery

ALGORITHM ShadowPaging(Transaction T)

BEGIN

// Two page tables are maintained: shadow (stable, on disk) and current (working copy)

shadowPageTable = COPY(CurrentPageTable) // snapshot before T begins

currentPageTable = CurrentPageTable // T operates only on this copy

TRY

FOR EACH write operation (page P, newData) IN T DO

newPage = ALLOCATE new disk block

WRITE newData TO newPage // never overwrite the old page in place

currentPageTable[P] = newPage.address // redirect pointer to new page

END FOR

// ---- COMMIT ----

FLUSH all modified pages referenced by currentPageTable TO disk

ATOMIC_WRITE(currentPageTable AS new shadowPageTable) // single pointer swap

DISCARD old shadowPageTable and orphaned old pages

RETURN COMMIT_SUCCESS

CATCH failure/crash DURING T

// ---- ROLLBACK ----

currentPageTable = COPY(shadowPageTable) // simply discard the working copy

FREE all newly allocated pages made by T

RETURN ROLLBACK_COMPLETE // no UNDO log processing needed

END TRY

END

- Because the shadow table always points to the last consistent, committed state, recovery after a crash just means reverting the page-table pointer — no REDO/UNDO log replay is required.

Q9. Immediate Update Recovery (UNDO/REDO Log Processing)

ALGORITHM ImmediateUpdateRecovery(LogFile L)

BEGIN

undoList = EMPTY LIST // transactions with BEGIN but no COMMIT

redoList = EMPTY LIST // transactions with COMMIT recorded

// Scan log from the most recent checkpoint forward

FOR EACH logRecord IN L (in order) DO

IF logRecord.type == START_TRANSACTION THEN

undoList.add(logRecord.transID)

ELSE IF logRecord.type == COMMIT THEN

undoList.remove(logRecord.transID)

redoList.add(logRecord.transID)

END IF

END FOR

// ---- UNDO phase: reverse order, only uncommitted transactions ----

FOR EACH logRecord IN L (REVERSE order) DO

IF logRecord.transID IN undoList AND logRecord.type == UPDATE THEN

WRITE(logRecord.dataItem, logRecord.oldValue) // restore old value

END IF

END FOR

// ---- REDO phase: forward order, only committed transactions ----

FOR EACH logRecord IN L (forward order) DO

IF logRecord.transID IN redoList AND logRecord.type == UPDATE THEN

WRITE(logRecord.dataItem, logRecord.newValue) // reapply new value

```

    END IF
  END FOR

  FLUSH all recovered pages TO disk
  RETURN DATABASE_CONSISTENT
END

```

- Immediate update means changes are written to the database buffer/disk before commit, so both UNDO (for aborted/incomplete transactions) and REDO (for committed ones lost from volatile buffers) are needed on recovery — unlike deferred update, which needs REDO only.

Q10. Database Connectivity Manager (Connection Pooling)

```

ALGORITHM ConnectivityManager(MaxPoolSize N)
BEGIN
  pool = INITIALIZE ConnectionPool WITH N pre-opened connections
  inUse = EMPTY SET

  FUNCTION GetConnection()
    IF pool.available() > 0 THEN
      conn = pool.remove()
    ELSE IF (pool.total() < MaxPoolSize) THEN
      conn = OpenNewConnection()
    ELSE
      WAIT until a connection is released, OR timeout -> RAISE PoolExhausted
    END IF
    inUse.add(conn)
    RETURN conn
  END FUNCTION

  FUNCTION ExecuteQuery(sql, params)
    conn = GetConnection()
    TRY
      conn.beginTransaction()

```

```

    stmt = conn.prepare(sql, params)      // parameterised, avoids injection
    result = stmt.execute()
    conn.commit()
    RETURN result
CATCH SQLException AS e
    conn.rollback()                      // exception rollback
    LOG(e)
    RAISE QueryExecutionError(e)
FINALLY
    ReleaseConnection(conn)              // always return to pool
END TRY
END FUNCTION

FUNCTION ReleaseConnection(conn)
    inUse.remove(conn)
    IF conn.isValid() THEN
        pool.add(conn)                  // recycle for reuse
    ELSE
        conn.close()
        pool.add(OpenNewConnection())    // replace broken connection
    END IF
END FUNCTION
END

```